Excepciones

Una excepción en programación es un evento o problema que ocurre durante la ejecución de un programa y altera su flujo normal de instrucciones.

Causas comunes de una excepción:

- División por cero (operación matemática no válida).
- Acceso a un índice inexistente en un arreglo o vector.
- Invocación de un método sobre un objeto nulo (null).
- Apertura de un archivo que no existe o falta de permisos en el sistema.

Manejo de excepciones

El manejo de excepciones es el mecanismo que nos permite interceptar estos errores mediante bloques de código especiales (como try, catch). Su objetivo es evitar que el software se detenga de golpe (cierre abrupto), permitiendo registrar el error, mostrar un mensaje amigable al usuario o intentar recuperar el estado del sistema para que siga funcionando.

Supongamos el siguiente código:

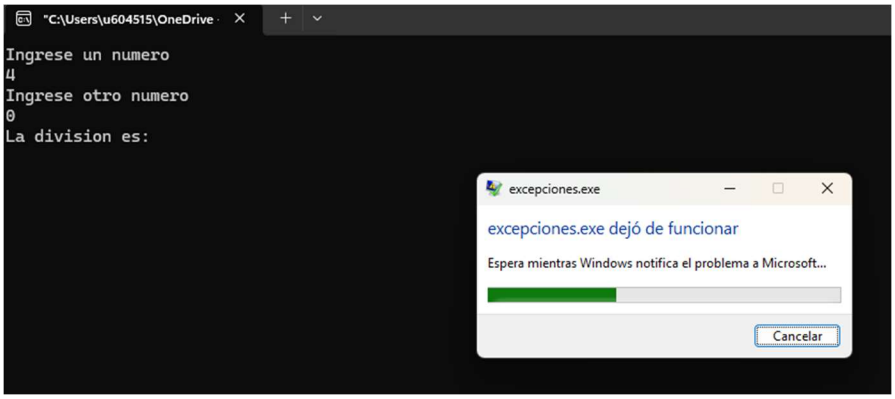
```cpp
#include <iostream>

using namespace std;

int dividir(int a,int b)
{
    return a/b;
}

int main()
{
    int numero1,numero2;
    cout<<"Ingrese un numero"<<endl;
    cin>>numero1;
    cout<<"Ingrese otro numero"<<endl;
    cin>>numero2;
    cout<<"La division es: "<<dividir(numero1,numero2)<<endl;
    return 0;
}
```

Nada impide al usuario ingresar un 0 como segundo número, lo cual al intentar realizar el cálculo obtenemos un error en ejecución.

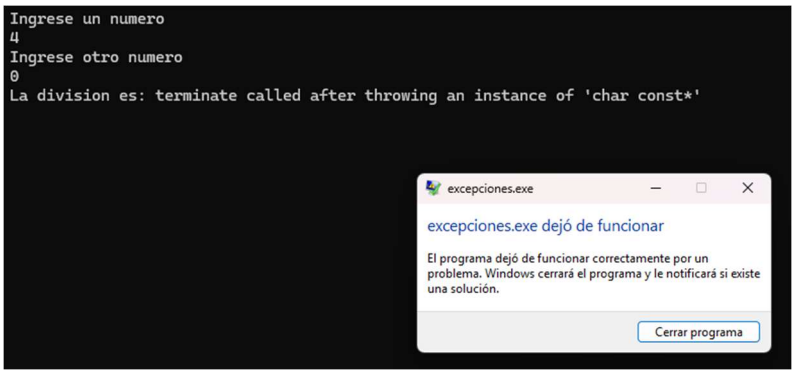


Con lo aprendido hasta ahora podríamos verificar al ingresar el segundo número que no sea 0, o lo mismo podemos hacer dentro de la función, pero deberíamos mostrar un mensaje dentro de ella. Esto queda medio desprolijo y para el primer caso contamos con una función y obligamos a quien la utilice se tenga que acordar de realizar esa verificación.

Para evitar estas desprolijidades utilizaremos excepciones:

```
int dividir(int a,int b)
{
    if(b==0)
        throw "No se puede dividir por 0";
    return a/b;
}
```

Dentro de la función creamos una validación de la segunda variable, y si esta es igual a 0 lanzamos mediante throw, una excepción. El lanzamiento de una excepción es un mecanismo que nos permite evitar la interrupción de la línea de ejecución de un programa. Si ejecutamos nuevamente este código nos encontramos con:



Lo que sucede es que no tenemos a nadie que administre o capture esa excepción y sepa que tenga algo para hacer.

Para eso utilizamos try y catch:

```
int dividir(int a,int b)
{
    if(b==0)
        throw "No se puede dividir por 0";
    return a/b;
}

int main()
{
    int numero1,numero2;
    cout<<"Ingrese un numero"<<endl;
    cin>>numero1;
    cout<<"Ingrese otro numero"<<endl;
    cin>>numero2;
    try{
        cout<<"La division es: "<<dividir(numero1,numero2)<<endl;
    }
    catch(const char*e){
        cout<<e<<endl;
    }

    return 0;
```

Desglose de cada palabra clave:

- **throw** → lanza (levanta) una excepción. Corta inmediatamente el flujo normal, no importa en qué punto de la función esté.
- **try** → delimita el bloque donde se van a vigilar excepciones
- **catch** → captura una excepción de un tipo específico y la maneja

Cuando se lanza una excepción, la ejecución no vuelve al lugar del throw. Salta directo al primer catch compatible, saltándose todo el código intermedio, sin importar cuántas funciones haya en el medio. Esto es justo lo que resuelve el problema de "propagar el error a mano nivel por nivel".

Qué se puede lanzar (y qué conviene lanzar)

Técnicamente se puede lanzar cualquier tipo: un int, un char*, un objeto propio. Pero:

```
throw 404;              // funciona, pero no dice nada
throw "algo fallo";     // funciona, pero es solo texto
throw MiExcepcion(...); // la forma correcta: un objeto
```

Lanzar siempre objetos, nunca literales sueltos. Un objeto puede llevar datos adicionales (qué pila falló, qué índice era inválido) y participar de una jerarquía de tipos.

La jerarquía estándar: std::exception

C++ ya trae una jerarquía de excepciones en <stdexcept>.

```
std::exception
    ├── std::logic_error
    │       ├── std::invalid_argument
    │       ├── std::out_of_range
    │       └── std::length_error
    └── std::runtime_error
            ├── std::overflow_error
            └── std::underflow_error
```

```
#include <stdexcept>

int obtener(int arreglo[], int n, int indice) {
    if (indice < 0 || indice >= n) {
        throw out_of_range("Indice fuera de rango");
    }
    return arreglo[indice];
}

try {
    int v = obtener(datos, 5, 10);
}
catch (const out_of_range& e) {
    cout << "Error: " << e.what() << endl;
}
```

Multiples catch y un catch genérico

```
try {
    // ...
}
catch (const out_of_range& e) {
    cout << "Fuera de rango: " << e.what() << endl;
}
catch (const invalid_argument& e) {
    cout << "Argumento invalido: " << e.what() << endl;
}
catch (const exception& e) {
    cout << "Otro error de la jerarquia estandar: " << e.what() << endl;
}
catch (...) {
    cout << "Excepcion desconocida" << endl;
}
```

Dos reglas para remarcar:

El orden importa. Los catch se evalúan de arriba hacia abajo y se ejecuta el primero que coincida. Si catch (const exception&) va antes que catch (const out_of_range&), el segundo nunca se alcanza (porque out_of_range es un exception, el primero ya lo captura).

catch (...) captura cualquier cosa, incluso tipos que no heredan de exception. Va siempre al final, como red de seguridad.

Excepciones propias

```
#include <exception>
#include <string>

class PilaVaciaException : public exception {
private:
    string mensaje;
public:
    PilaVaciaException(const string& msg = "La pila esta vacia")
        : mensaje(msg) {}

    const char* what() const noexcept override {
        return mensaje.c_str();
    }
};
```

Como utilizarlo:

```
template <typename T>
T Pila<T>::desapilar() {
    if (estaVacia()) {
        throw PilaVaciaException();
    }
    tope--;
    return elementos[tope];
}
```